

Week 7 Problem Set

Jolie Barga <https://github.com/joliebarga/soc10problemsets>

May 19 2026

1. Start a new document, `problemset7.Rmd`, in your `soc10problemsets` repository. (You can download a template here, but be sure to save it in your own `soc10problemsets` repo). Create a map of the `ames` data, but change the cluster specifications to try to get closer to the “true” neighborhoods shown here. Describe the benefits and limitations of your approach. **Commit** your changes.

I modified the k-means clustering algorithm to use 28 cluster centers, which matches the number of neighborhoods defined in the `ames` dataset. By increasing the number of centers from the default of 3 or 6, the algorithm is able to identify geographic distinctions between groups of houses, producing a map that more closely resembles the neighborhood boundaries shown in the class notes. The main benefit of k-means clustering is that it is simple, fast, and does not require any prior knowledge of what the neighborhoods actually look like or where their boundaries are because the algorithm finds these groupings on its own. However, there are several important limitations to this approach. First, k-means assumes that clusters are about equal in size, which may not reflect the true shape or population density of real neighborhoods in Ames. Second, because k-means starts with randomly assigned centroids, the results can vary from run to run. Third, this model only uses longitude and latitude to form clusters, which means it ignores other features that likely define real neighborhood boundaries, such as housing prices, building types, school districts, or historical development patterns which are important considerations as well.

```
library(tidymodels)
library(sf)
library(magrittr)

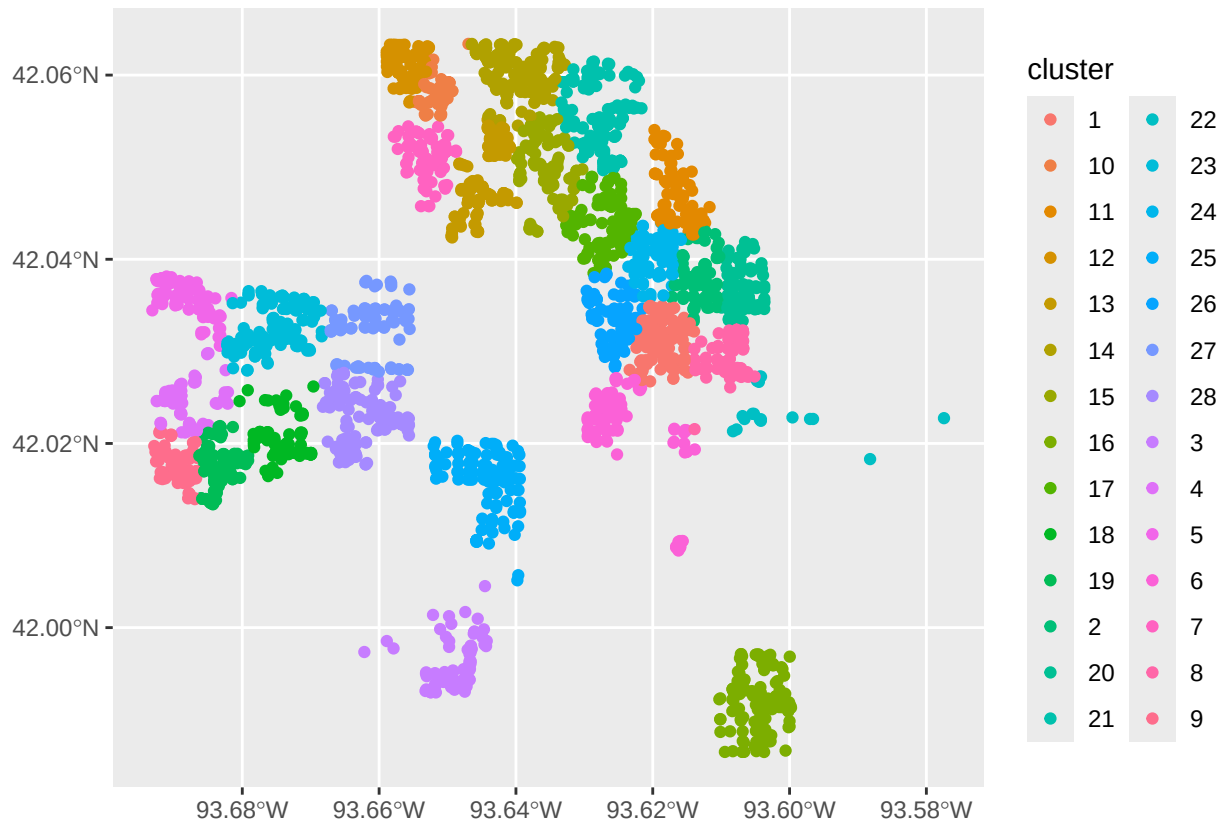
# load ames data
data(ames)

# make ames spatial
ames_spatial <- ames %>%
  st_as_sf(coords = c("Longitude", "Latitude"), crs = 4326)

# k-means clustering
kclust <- kmeans(ames %>%
  select(Longitude, Latitude), centers = 28)

# add clusters
ames_spatial %<>%
  mutate(cluster = as.character(kclust$cluster))

ggplot(ames_spatial) +
  geom_sf(aes(col = cluster))
```



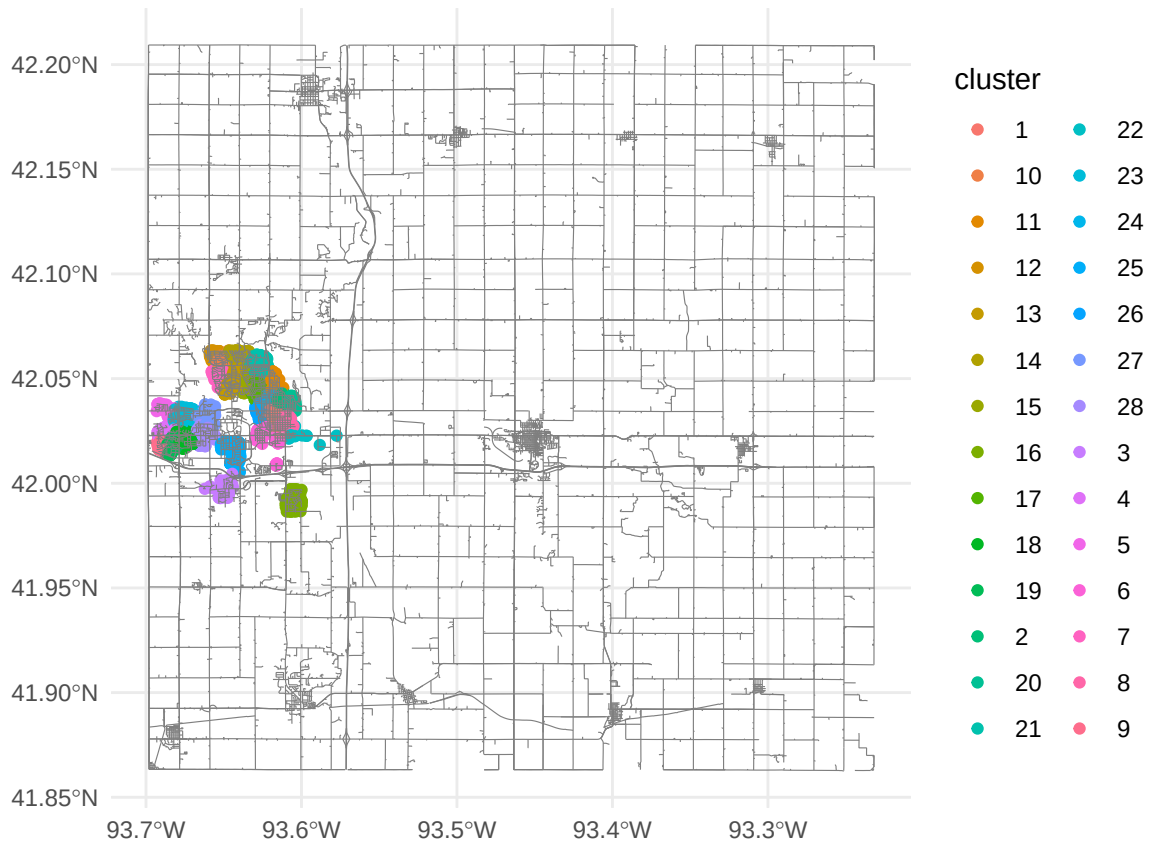
2. Using the `tigris` package, download roads data for the county of Story, IA. Add roads to your plot. **Commit** your changes.

```
library(tigris)
library(sf)
library(ggplot2)
```

```
# download roads data for Story County, IA
story_roads <- roads("IA", "Story")
```

```
## |
```

```
# plot with roads added
ggplot() +
  geom_sf(data = ames_spatial, aes(col = cluster)) +
  geom_sf(data = story_roads, color = "gray50", linewidth = 0.2) +
  theme_minimal()
```



3. Try to train an algorithm to predict housing sales prices. Create a random forest to predict the sales price, using some new or different variables than the model above. Create a model that *better* predicts sales price, and explain why it is better. **Commit** your changes.

I added four new variables to the random forest model: Overall_Cond, Total_Bsmt_SF, Garage_Cars, and Full_Bath. These variables capture important aspects of a home's quality and size that the original model we went through in class did not include. Overall condition rating reflects how buyers perceive the home's upkeep, basement size and garage capacity capture structural features that strongly influence price, and the number of full bathrooms is a key amenity that buyers consider when deciding on a house. On the training set, my model achieved an rmse of 12,647 and an rsq of 0.977, compared to the class model's rmse of 15,364 and rsq of 0.966. On the test set, my model achieved an rmse of 33,486 and an rsq of 0.851, compared to the class model's rmse of 33,329 and rsq of 0.823. The higher rsq on the test set indicates that my model explains more variance in housing prices and generalizes better to unseen data, making it a better predictive model overall.

```
library(tidymodels)

# split data
ames_split <- initial_split(ames, prop = 0.80, strata = Sale_Price)
ames_train <- training(ames_split)
ames_test <- testing(ames_split)

# create a random forest
rf_model <-
  rand_forest(trees = 1000) %>%
```

```

set_engine("ranger") %>%
set_mode("regression")

# define the random forest workflow
rf_wflow <-
  workflow() %>%
  add_formula(
    Sale_Price ~ Gr_Liv_Area + Year_Built + Bldg_Type + Neighborhood +
      Latitude + Longitude + Overall_Cond + Total_Bsmt_SF +
      Garage_Cars + Full_Bath) %>%
  add_model(rf_model)

# fit the random forest model
rf_fit <- rf_wflow %>% fit(data = ames_train)

estimate_perf <- function(model, dat) {
  # Capture the names of the `model` and `dat` objects
  cl <- match.call()
  obj_name <- as.character(cl$model)
  data_name <- as.character(cl$dat)
  data_name <- gsub("ames_", "", data_name)

  # Estimate these metrics:
  reg_metrics <- metric_set(rmse, rsq)

  # output our model
  output <- model %>%
    predict(dat) %>%
    bind_cols(dat %>% select(Sale_Price)) %>%
    reg_metrics(Sale_Price, .pred) %>%
    select(-.estimator) %>%
    mutate(object = obj_name, data = data_name)

  return(output)
}

# examine random forest performance
estimate_perf(rf_fit, ames_train)

```

```

## # A tibble: 2 x 4
##   .metric .estimate object data
##   <chr>      <dbl> <chr>  <chr>
## 1 rmse    12906.   rf_fit train
## 2 rsq      0.976   rf_fit train

```

```

# final rf model
final_rf_res <- last_fit(rf_wflow, ames_split)

# get model metrics
collect_metrics(final_rf_res)

```

```

## # A tibble: 2 x 4
##   .metric .estimator .estimate .config

```

```
##   <chr>   <chr>           <dbl> <chr>
## 1 rmse    standard    25405.  pre0_mod0_post0
## 2 rsq     standard      0.901  pre0_mod0_post0
```

4. Perhaps the model could be better trained using Cross Validation. Set up a 10-fold cross validation model, and predict sales prices again. Compare these results to your prior results. Are they similar or different, and what does this mean? **Commit** your changes.

From a 10-fold cross validation with the same random forest model from Q3, the model achieved an average rmse of 26,890 and an average rsq of 0.886 across the 10 folds. I can compare these results to the test set results from Q3 which had an rmse of 33,486 and an rsq of 0.851 so the cross validated model performs better on both metrics. This makes sense because cross validation repeatedly trains and evaluates the model on different slices of the training data, which helps reduce overfitting and produces a more stable estimate of model performance. The fact that the cross validated rmse is lower and rsq is higher than the test set suggests that the model generalizes good and that cross validation is a useful tool for getting a more reliable sense of how the model will perform on new data. These results are also consistent with what the class notes found, where cross validation performance was similar to test set performance, confirming that our model is not drastically overfitting to the training data.

```
# set up cross validation
ames_folds <- vfold_cv(ames_train, v = 10)

keep_pred <- control_resamples(save_pred = TRUE, save_workflow = TRUE)

# create resample object
rf_res <-
  rf_wflow %>%
  fit_resamples(resamples = ames_folds, control = keep_pred)

# examine model metrics
collect_metrics(rf_res)
```

```
## # A tibble: 2 x 6
##   .metric .estimator   mean     n std_err .config
##   <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1 rmse    standard    27129.     10 1107.   pre0_mod0_post0
## 2 rsq     standard      0.885     10  0.0110 pre0_mod0_post0
```

5. If a real estate company used algorithms to determine home values, could this have any societal implications? Walk the reader through why the data could induce biases and how you might try to resolve these potential problems. **Commit** your changes.

If a real estate company used algorithms to determine home values, this could have significant societal implications because if we look at how the ames dataset includes variables like Neighborhood, Latitude, and Longitude these are geographic features that can act as proxies for race and socioeconomic status due to historical patterns of residential segregation. If an algorithm is trained on historical sales data that already reflects these inequalities, it may learn to assign lower values to homes in certain neighborhoods not because of the physical characteristics of the homes but because of who lives there, which would effectively encode existing housing discrimination into the system and make it harder for residents of these neighborhoods to build wealth through homeownership.

Additionally, variables like Garage_Cars, Total_Bsmt_SF, and overall condition ratings may systematically disadvantage older housing stock that is more common in certain neighborhoods, further compounding these

biases in predicted home values. As Molina and Garip note and I learned in CS 186, even if a researcher removes an explicitly biased variable like neighborhood from the model, the algorithm may still use other variables as proxies that capture the same underlying inequalities which makes this a difficult problem to fully solve. One potential way to reduce these biases would be to remove geographic variables like Neighborhood, Latitude, and Longitude from the model entirely and focus only on the physical characteristics of the home. However this may not fully resolve the problem because other variables can still act as proxies for location. A more comprehensive approach might involve auditing the model's predictions across different demographic groups and geographic areas to identify where systematic disparities exist and then adjusting the model to ensure more equitable outcomes through a trial and error method and if ever used in real housing data checking frequently and adjusting when needed to ensure no biases with the help of researchers and data scientists.